

SMART ROAD VISION SYSTEM (SRVS V4) — FRONTEND DESIGN & DEVELOPMENT MASTER PLAN

This master plan details the layout design, lifecycle architecture, and data orchestration layers required to build the rest of the frontend dashboard around the `VideoAnalyticsStation.tsx` core component.

1. Complete Layout Grid Blueprint & Multi-Route Architecture

The dashboard uses a dark terminal theme built on a **responsive grid layout**. The system supports client-side view switching to keep performance fast and prevent full page refreshes.

Navigation Hierarchy

- `[Dashboard Hub]` : Main workspace comprising the dual video analytical feed streams, dynamic live logging, and the SQLite ledger.
- `[Snapshot Gallery]` : Interactive high-resolution archive of captured vehicles and their Real-ESRGAN micro-enhanced variations with OCR diagnostics.
- `[CSV Ledger Explorer]` : A dual-pane environment for auditing CSV records compiled by the backend processing pipelines.

[SRVS V4] AI VIDEO MONITORING STATION

Karachi Hub Desk

└─ [ Monitor Hub]

└─ [ Enhanced Gallery]

└─ [ CSV Records Explorer]

2. Page & Subsystem Detail Specifications

Section A: Main Monitor Workspace (/dashboard)

Includes the layout detailed in the primary blueprint, with the:

- **Session Control Panel:** Dynamic selectors for front/rear plate video clips, simulation toggle, processing cadence state.
- **Zone 1 (Core Video Stations):** Real-time 60Hz canvas analytical overlays powered by the synchronized timeline loop.
- **Zone 2 (Live Telemetry Log):** Real-time scrolling ticker feed printing detections, safety warnings, and database ingestion ticks.
- **Zone 3 (Enforcement Ledgers):** Multi-table SQLite database ledger with tab switching (Motorcycles , Rickshaws , Large Vehicles) and regex-based instantaneous filtering on detected license plate strings.

Section B: Enhanced Snapshots Route (/snapshots) [NEW REQUIREMENT]

This section isolates the evidentiary images produced by the high-speed camera sensors and details the deep-learning enhancement process:

1. Snapshots Collection Grid:

- Displays vehicle snapshots side-by-side with their **Real-ESRGAN** super-resolved counterparts as interactive, hover-scaled thumbnails.
- Distinct tags showing vehicle class (Motorcycle , Auto-rickshaw , etc.) and processing flags (Real-ESRGAN Optimized , Direct Crop).

2. Full-Screen Interactive Inspection Console:

- Clicking a snapshot triggers a beautiful analytical overlay.
- **Floating Diagnostics Board (Top-Right):** Displays the final license plate read out, color-coded by the generation method:
 - **Local OCR (Tesseract / PaddleOCR):** Displayed with raw structural confidence metrics (e.g., Confidence: 94.2%).

- **API Fallback (Gemini API Cloud Gateway):** Displayed when local OCR falls below confidence thresholds, showing processing status (e.g., `Model: Gemini-2.5-Flash Cloud`).
- **Interactive Comparison Lens Slider:** An overlay with a split dragging slider showing a live "before/after" comparison. Dragging reveals the raw low-resolution pixel values versus the razor-sharp enhanced plate characters.

Section C: CSV Ledger Explorer (`/csv-explorer`) [NEW REQUIREMENT]

A flat-file analytical interface for viewing compiled tabular databases:

1. Left-Hand Control Pane:

- Lists all dynamically active `.csv` files stored in the output pipeline (e.g., `motorcycle_violations.csv`, `rickshaws_unregistered_log.csv`, `heavy_vehicle_velocities.csv`, `api_fallback_telemetry.csv`).
- Displays file metadata: total record counts, file size, and last system synchronization time.

2. Right-Hand SQL-Like Data Grid:

- Selecting a file instantly parses and loads its records into an interactive grid layout.
- Includes real-time client-side search modifiers, sorting indexes for speed columns, class filter toggles, and violation flags highlighted in neon red and warning amber.

3. Database Ingestion & Telemetry Ingestion Loop

As your python background scripts populate SQLite and CSV logs, the client utilizes an optimized frame lookup schema:

$O(1)$ Frame Lookup Strategy

Instead of evaluating rows through linear array filters at $O(N)$ runtime per frame render (which ruins 60 FPS performance), the system parses incoming database records directly into a hash map indexed by the timeline frame:

```
export interface TrackData {
  track_id: number;
  bbox: [number, number, number, number]; // [x1, y1, x2, y2]
  class_name: string;
  speed: number;
  violation: boolean;
  plate?: string;
  ocr_method?: 'local' | 'api_fallback';
}

export interface TelemetryMap {
  [frameIndex: number]: TrackData[];
}
```

4. Development & Verification Milestones

- [] **Phase 1: Ingestion Infrastructure Setup:** Create Next.js server targets or mock providers matching active SQLite entries.
- [] **Phase 2: Video Overlay and Scale Transformation:** Map YOLO frame bounding boxes smoothly onto viewport dimensions.
- [] **Phase 3: Deep-Learning Comparison Deck:** Construct the drag-lens `clip-path` component for Real-ESRGAN visualization.